SENATI
Curso: Inteligencia Artificial

Guía 02

Uso de las librerías Scikit-learn y PyTorch

Aprendizaje automático clásico y redes neuronales en Visual Studio Code

Dato	Detalle
Instructor	Rembrandt Ubalde Enríquez
Estudiante	_____
Fecha	____ / ____ / 2026
Duración estimada	6 horas pedagógicas
Herramientas	Python 3, Visual Studio Code, Scikit-learn, PyTorch, NumPy, Pandas

Guía práctica paso a paso con Visual Studio Code

Contenido

- Presentación de la guía
- Parte 1. Preparación del entorno en Visual Studio Code
- Parte 2. Aprendizaje automático con Scikit-learn
- Parte 3. Redes neuronales con PyTorch
- Parte 4. Errores frecuentes y cómo resolverlos
- Parte 5. Ejercicios propuestos
- Lista de cotejo (autoevaluación)

Presentación de la guía

Objetivo general

Estudiar y aplicar las librerías **Scikit-learn** y **PyTorch** para construir, entrenar, evaluar y guardar modelos de inteligencia artificial, trabajando en Visual Studio Code.

Objetivos específicos

- Preparar un proyecto en VS Code con Scikit-learn y PyTorch instalados.
- Cargar un dataset, dividirlo en entrenamiento y prueba, y escalar sus características.
- Entrenar modelos de clasificación y regresión con Scikit-learn y evaluarlos con métricas.
- Comparar modelos con Pipeline y validación cruzada, y guardar el mejor modelo.
- Manejar tensores y el cálculo automático de derivadas (autograd) en PyTorch.
- Programar el bucle de entrenamiento de una red neuronal y evaluarla.

Conceptos clave

Concepto	Significado
Aprendizaje supervisado	El modelo aprende de ejemplos que ya tienen la respuesta correcta (etiqueta).
Características (X)	Datos de entrada del modelo, por ejemplo medidas de una flor.
Etiqueta (y)	Lo que se quiere predecir, por ejemplo la especie de la flor.
Clasificación / Regresión	Clasificación predice una categoría; regresión predice un número.
Entrenamiento / Prueba	Con los datos de entrenamiento el modelo aprende; con los de prueba se mide qué tan bien generaliza.
Tensor	Arreglo de n dimensiones de PyTorch, similar a un arreglo de NumPy, que puede usar la GPU y calcular gradientes.
Época	Una pasada completa del modelo por todos los datos de entrenamiento.
Función de pérdida	Mide el error del modelo. El entrenamiento busca hacerla lo más pequeña posible.

Librería	Enfoque	Cuándo usarla
Scikit-learn	Aprendizaje automático clásico (árboles, KNN, regresiones, SVM, clustering).	Datos en tablas, proyectos rápidos, modelos explicables, preprocesamiento y métricas.
PyTorch	Aprendizaje profundo (redes neuronales) con tensores y GPU.	Imágenes, texto, audio, modelos grandes y redes personalizadas.

Requisitos previos

- Haber completado la **Guía 01 (NumPy y Pandas)**.

- Al menos 3 GB libres en disco (PyTorch es una librería grande).

Parte 1. Preparación del entorno en Visual Studio Code

En esta parte dejarás listo tu equipo para programar en Python dentro de Visual Studio Code (VS Code). Si ya lo hiciste en una guía anterior, revisa rápidamente cada paso y continúa desde el **Paso 4** creando una carpeta nueva para esta guía.

Paso 1. Instalar Python

1. Ingresa a **python.org/downloads** y descarga la versión estable más reciente de Python 3 (3.11 o superior).
2. Ejecuta el instalador. En la primera pantalla **marca la casilla "Add python.exe to PATH"** y luego pulsa **Install Now**.
3. Al terminar, abre el **Símbolo del sistema** (cmd) y verifica la instalación con los comandos de abajo.

Símbolo del sistema (Windows)

```
python --version
pip --version
```

Resultado esperado en la terminal

```
Python 3.12.x
pip 24.x from C:\Users\...\site-packages\pip (python 3.12)
```

Si el comando python no funciona

Mensaje "No se encontró Python; ejecutar sin argumentos para instalar desde Microsoft Store...": Python no está instalado o no está en el PATH, y Windows redirige el comando a la Store. Para solucionarlo:
1. Instala Python desde **python.org** marcando **Add python.exe to PATH**. Sin permisos de administrador, desmarca **Use admin privileges when installing py.exe**. También puedes usar en la terminal: `winget install Python.Python.3.12`
2. Abre **Configuración > Aplicaciones > Configuración avanzada de aplicaciones > Alias de ejecución de aplicaciones** y desactiva **python.exe** y **python3.exe** (Instalador de aplicación).
3. **Cierra VS Code por completo y vuelve a abrirlo**: una terminal ya abierta conserva el PATH anterior.
4. Verifica con `python --version` o con `py --version`.

Mensaje "python no se reconoce como un comando interno": Python se instaló sin agregarse al PATH. Vuelve a ejecutar el instalador, elige **Modify** y activa **Add Python to environment variables**.

Paso 2. Instalar Visual Studio Code y sus extensiones

1. Descarga VS Code desde **code.visualstudio.com** e instálalo. Durante la instalación marca **"Agregar la acción Abrir con Code"** y **"Agregar a PATH"**.
2. Abre VS Code y entra al panel de **Extensiones** (ícono de cuadritos en la barra izquierda o `Ctrl + Shift + X`).
3. Busca e instala la extensión **Python** (publicada por Microsoft). Se instalará también **Pylance**, que da autocompletado y detección de errores.
4. Instala la extensión **Jupyter** (Microsoft) para trabajar con cuadernos `.ipynb` dentro de VS Code.

5. Opcional: instala **Spanish Language Pack for Visual Studio Code** si prefieres los menús en español.

Paso 3. Conocer la interfaz de VS Code

Zona de VS Code	Para qué sirve
Explorador (`Ctrl + Shift + E`)	Muestra las carpetas y archivos del proyecto. Desde aquí creas archivos nuevos.
Editor	Zona central donde escribes el código. Puedes tener varias pestañas abiertas.
Terminal integrada (`Ctrl + ñ` o menú **Ver > Terminal**)	Ejecuta comandos (pip, python) sin salir de VS Code.
Paleta de comandos (`Ctrl + Shift + P`)	Busca cualquier acción de VS Code escribiendo su nombre.
Barra de estado (abajo)	Muestra el intérprete de Python activo, errores y advertencias.
Botón ▶ Run Python File (arriba a la derecha)	Ejecuta el archivo `.py` abierto y muestra el resultado en la terminal.

Paso 4. Crear la carpeta del proyecto

1. En el Explorador de Windows crea la carpeta **IA_Guia02_Sklearn_PyTorch** (por ejemplo, en Documentos).
2. En VS Code ve a **Archivo > Abrir carpeta...** y selecciona la carpeta creada.
3. Si aparece el mensaje **"¿Confía en los autores de los archivos de esta carpeta?"**, pulsa **Sí, confío en los autores**.

Paso 5. Crear y activar un entorno virtual

Un **entorno virtual** es una instalación de Python aislada solo para este proyecto. Evita conflictos entre versiones de librerías de distintos proyectos.

1. Abre la terminal integrada con **Ver > Terminal** (o `Ctrl + ñ`).
2. Escribe el comando para crear el entorno virtual llamado `venv`.
3. Actívalo. Sabrás que está activo porque al inicio de la línea aparece **(venv)**.

Terminal de VS Code

```
# Crear el entorno virtual
python -m venv venv

# Activarlo en Windows (PowerShell)
.\venv\Scripts\Activate.ps1

# Activarlo en Windows (cmd)
venv\Scripts\activate

# Activarlo en Linux / macOS
```

```
source venv/bin/activate
```

> **Importante**
> Si PowerShell muestra **"la ejecución de scripts está deshabilitada en este sistema"**, ejecuta una sola vez:
> `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned` y responde **S** (Sí). Luego vuelve a activar el entorno.

Paso 6. Seleccionar el intérprete de Python en VS Code

1. Presiona `Ctrl + Shift + P` y escribe **Python: Select Interpreter**.
2. Elige la opción que contiene **('venv': venv)** y la ruta `.\venv\Scripts\python.exe`.
3. Comprueba en la barra de estado (parte inferior derecha) que aparezca la versión de Python con **('venv')**.

Paso 7. Instalar las librerías de la guía

Con el entorno virtual activo, instala las librerías desde la terminal integrada:

Terminal de VS Code

```
python -m pip install --upgrade pip
pip install numpy pandas scikit-learn joblib matplotlib
pip install torch
```

Verifica que la instalación fue correcta:

Terminal de VS Code

```
python -c "import sklearn, torch; print(sklearn.__version__, torch.__version__)"
```

> **Sugerencia**
> Guarda la lista de librerías de tu proyecto con `pip freeze > requirements.txt`. Así un compañero puede instalar lo mismo con `pip install -r requirements.txt`.

Paso 8. Cómo crear y ejecutar cada archivo de esta guía

1. En el Explorador de VS Code pulsa el ícono **Nuevo archivo** (o `Ctrl + N` y luego `Ctrl + S`).
2. Escribe el nombre indicado en la guía con extensión `.py` (por ejemplo `01_ejemplo.py`) y presiona Enter.
3. Copia el código, guárdalo con `Ctrl + S` y ejecútalo con el botón ▶ **Run Python File** (arriba a la derecha) o escribiendo `python nombre_archivo.py` en la terminal.
4. Compara lo que aparece en la terminal con el recuadro verde **"Resultado esperado"**.
5. Para ejecutar solo algunas líneas, selecciónalas y presiona `Shift + Enter`: se ejecutarán en la ventana interactiva de Python.

Sobre la instalación de PyTorch

El comando `pip install torch` instala la versión para procesador (CPU), suficiente para esta guía.
Si tu equipo tiene una tarjeta gráfica NVIDIA, ingresa a **pytorch.org/get-started/locally**, elige tu sistema operativo, **Pip**, **Python** y la versión de **CUDA**, y copia el comando que te muestra la página.
La descarga puede tardar varios minutos. No cierres VS Code mientras la terminal muestre el avance.

Parte 2. Aprendizaje automático con Scikit-learn

Usaremos el dataset **Iris**, que viene incluido en Scikit-learn: 150 flores con 4 medidas (largo y ancho del sépalo y del pétalo) y 3 especies a predecir.

Paso 9. Cargar y explorar el dataset

Crea el archivo **01_sklearn_datos.py**.

Archivo: 01_sklearn_datos.py

```
# 01_sklearn_datos.py - Cargar y explorar un dataset de Scikit-learn
import sklearn
import pandas as pd
from sklearn.datasets import load_iris

print("Versión de scikit-learn:", sklearn.__version__)

iris = load_iris()
X = iris.data          # características (entradas)
y = iris.target        # etiquetas (salida a predecir)

print("Forma de X:", X.shape)
print("Características:", iris.feature_names)
print("Clases:", [str(n) for n in iris.target_names])

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)

# Convertir a DataFrame para explorarlo con Pandas
df = pd.DataFrame(X, columns=iris.feature_names)
df["especie"] = [iris.target_names[i] for i in y]
print(df.head())
print(df["especie"].value_counts())
```

Resultado esperado en la terminal

```
Versión de scikit-learn: 1.9.1
Forma de X: (150, 4)
Características: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width
(cm)']
Clases: ['setosa', 'versicolor', 'virginica']
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm) especie
0                5.1               3.5                1.4               0.2  setosa
1                4.9               3.0                1.4               0.2  setosa
2                4.7               3.2                1.3               0.2  setosa
```

```
3                4.6               3.1                1.5               0.2  setosa
4                5.0               3.6                1.4               0.2  setosa
especie
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
```

Paso 10. Flujo de trabajo de un modelo de Scikit-learn

Todos los modelos de Scikit-learn siguen los mismos pasos. Una vez que los aprendes, puedes cambiar de algoritmo modificando una sola línea.

Paso	Método	Descripción
1	`train_test_split()`	Separar los datos en entrenamiento y prueba.
2	`StandardScaler().fit_transform()`	Escalar las características para que estén en la misma escala.
3	`modelo = Algoritmo()`	Crear el modelo con sus parámetros (hiperparámetros).
4	`modelo.fit(X_train, y_train)`	Entrenar: el modelo aprende de los datos.
5	`modelo.predict(X_test)`	Predecir con datos que el modelo nunca vio.
6	`accuracy_score()`, `confusion_matrix()`	Evaluar comparando predicciones con las respuestas reales.

Paso 11. Entrenar y evaluar un clasificador (KNN)

Crea **02_sklearn_clasificacion.py**. El algoritmo **K vecinos más cercanos (KNN)** clasifica una flor según las especies de las 5 flores más parecidas.

Archivo: 02_sklearn_clasificacion.py

```
# 02_sklearn_clasificacion.py - Entrenar y evaluar un clasificador
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

X, y = load_iris(return_X_y=True)

# 1. Dividir en entrenamiento (80%) y prueba (20%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print("Entrenamiento:", X_train.shape, "| Prueba:", X_test.shape)

# 2. Escalar las características (media 0, desviación 1)
escalador = StandardScaler()
X_train_esc = escalador.fit_transform(X_train)   # aprende y transforma
X_test_esc = escalador.transform(X_test)         # solo transforma

# 3. Crear y entrenar el modelo
modelo = KNeighborsClassifier(n_neighbors=5)
```

```
modelo.fit(X_train_esc, y_train)

# 4. Predecir y evaluar
y_pred = modelo.predict(X_test_esc)
print("Exactitud (accuracy):", round(accuracy_score(y_test, y_pred), 3))
print("Matriz de confusión:\n", confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred,
      target_names=["setosa", "versicolor", "virginica"]))

# 5. Predecir una flor nueva
flor_nueva = [[5.1, 3.5, 1.4, 0.2]]
pred = modelo.predict(escalador.transform(flor_nueva))
print("Especie predicha para la flor nueva:", pred[0])
```

Resultado esperado en la terminal

```
Entrenamiento: (120, 4) | Prueba: (30, 4)
Exactitud (accuracy): 0.933
Matriz de confusión:
 [[10  0  0]
 [ 0 10  0]
 [ 0  2  8]]
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.83      1.00      0.91        10
   virginica       1.00      0.80      0.89        10

    accuracy                           0.93        30
   macro avg       0.94      0.93      0.93        30
weighted avg       0.94      0.93      0.93        30

Especie predicha para la flor nueva: 0
```

Cómo leer los resultados

Exactitud 0.933: el modelo acertó 28 de 30 flores de prueba (93.3 %).
Matriz de confusión: las filas son la especie real y las columnas la predicha. La diagonal son aciertos; aquí 2 flores *virginica* se confundieron con *versicolor*.
precision: de las que el modelo dijo que eran de una clase, cuántas lo eran. **recall**: de las que realmente eran de una clase, cuántas encontró.
La predicción **0** corresponde a la clase *setosa* (0 = setosa, 1 = versicolor, 2 = virginica).
Importante: el escalador se ajusta (`fit`) **solo** con los datos de entrenamiento, para no “filtrar” información de la prueba.

Paso 12. Comparar modelos con Pipeline y validación cruzada

Crea **03_sklearn_comparar.py**. Un **Pipeline** une el escalado y el modelo en un solo objeto. La **validación cruzada** entrena y evalúa 5 veces con distintas particiones para obtener un resultado más confiable.

Archivo: 03_sklearn_comparar.py

```
# 03_sklearn_comparar.py - Comparar modelos con Pipeline y validación cruzada
from sklearn.datasets import load_iris
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
import joblib

X, y = load_iris(return_X_y=True)

modelos = {
    "Regresión logística": make_pipeline(StandardScaler(),
LogisticRegression(max_iter=200)),
    "Árbol de decisión": DecisionTreeClassifier(max_depth=3, random_state=42),
    "KNN (k=5)": make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5)),
}

# Validación cruzada de 5 particiones
for nombre, modelo in modelos.items():
    puntajes = cross_val_score(modelo, X, y, cv=5)
    print(f"{nombre:22s} -> promedio: {puntajes.mean():.3f}  (+/-
{puntajes.std():.3f})")

# Entrenar el mejor modelo con todos los datos y guardarlo
mejor = modelos["Árbol de decisión"].fit(X, y)
joblib.dump(mejor, "modelo_iris.joblib")
print("Modelo guardado en modelo_iris.joblib")

# Cargar el modelo guardado y usarlo
cargado = joblib.load("modelo_iris.joblib")
print("Predicción con el modelo cargado:", cargado.predict([[6.7, 3.0, 5.2, 2.3]]))
```

Resultado esperado en la terminal

```
Regresión logística    -> promedio: 0.960  (+/- 0.039)
Árbol de decisión      -> promedio: 0.973  (+/- 0.025)
KNN (k=5)              -> promedio: 0.960  (+/- 0.025)
Modelo guardado en modelo_iris.joblib
Predicción con el modelo cargado: [2]
```

Explicación

El **árbol de decisión** obtuvo el mejor promedio (0.973), por eso se entrena con todos los datos y se guarda.
`joblib.dump` guarda el modelo entrenado en un archivo; `joblib.load` lo recupera sin volver a entrenar. Verás **modelo_iris.joblib** en el Explorador de VS Code.
La predicción **[2]** indica que la flor nueva es *virginica*.

Paso 13. Regresión lineal: predecir un número

Crea **04_sklearn_regresion.py**. Predeciremos la nota de un estudiante a partir de sus horas de estudio (datos simulados).

Archivo: 04_sklearn_regresion.py

```
# 04_sklearn_regresion.py - Regresión lineal: predecir un valor numérico
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, r2_score

# Datos simulados: horas de estudio vs. nota obtenida
rng = np.random.default_rng(0)
horas = rng.uniform(1, 10, size=50).reshape(-1, 1)
nota = 6 + 1.3 * horas.ravel() + rng.normal(0, 1, size=50)

X_train, X_test, y_train, y_test = train_test_split(horas, nota, test_size=0.2,
random_state=0)

reg = LinearRegression()
reg.fit(X_train, y_train)

print("Pendiente (coef):", round(reg.coef_[0], 3))
print("Intercepto:", round(reg.intercept_, 3))

y_pred = reg.predict(X_test)
print("Error absoluto medio (MAE):", round(mean_absolute_error(y_test, y_pred), 3))
print("Coeficiente R2:", round(r2_score(y_test, y_pred), 3))
print("Nota estimada con 8 horas de estudio:", round(reg.predict([[8]])[0], 2))
```

Resultado esperado en la terminal

```
Pendiente (coef): 1.375
Intercepto: 5.634
Error absoluto medio (MAE): 0.826
Coeficiente R2: 0.946
Nota estimada con 8 horas de estudio: 16.64
```

Explicación

El modelo aprendió la recta **nota ≈ 1.375 × horas + 5.634**: cada hora extra de estudio sube la nota en aproximadamente 1.4 puntos.
MAE 0.826: en promedio, la predicción se equivoca por menos de 1 punto. **R² 0.946**: el modelo explica cerca del 95 % de la variación de las notas (1.0 sería perfecto).

Parte 3. Redes neuronales con PyTorch

Paso 14. Tensores y autograd

Crea **05_pytorch_tensores.py**. Los tensores son la estructura de datos de PyTorch. Su gran ventaja es que pueden calcular derivadas automáticamente, que es lo que permite entrenar redes neuronales.

Archivo: 05_pytorch_tensores.py

```
# 05_pytorch_tensores.py - Tensores y autograd en PyTorch
import torch

print("Versión de PyTorch:", torch.__version__)
dispositivo = "cuda" if torch.cuda.is_available() else "cpu"
print("Dispositivo disponible:", dispositivo)

# 1. Crear tensores
t1 = torch.tensor([1.0, 2.0, 3.0])
t2 = torch.zeros(2, 3)
t3 = torch.rand(2, 2)            # valores aleatorios entre 0 y 1
print(t1, t1.shape, t1.dtype)
print(t2)

# 2. Operaciones
a = torch.tensor([[1., 2.], [3., 4.]])
b = torch.tensor([[5., 6.], [7., 8.]])
print("Suma:\n", a + b)
print("Producto matricial:\n", a @ b)
print("Media de a:", a.mean().item())

# 3. Conversión entre NumPy y PyTorch
import numpy as np
arr = np.array([1, 2, 3])
t_desde_np = torch.from_numpy(arr)
print("De NumPy a tensor:", t_desde_np)
print("De tensor a NumPy:", t_desde_np.numpy())

# 4. Autograd: cálculo automático de derivadas
x = torch.tensor(3.0, requires_grad=True)
y = x ** 2 + 2 * x + 1          # y = x² + 2x + 1
y.backward()                     # calcula dy/dx
print("Derivada de y en x=3:", x.grad.item())   # 2x + 2 = 8
```

Resultado esperado en la terminal

```
Versión de PyTorch: 2.x.x+cpu
Dispositivo disponible: cpu
tensor([1., 2., 3.]) torch.Size([3]) torch.float32
tensor([[0., 0., 0.],
        [0., 0., 0.]])
Suma:
 tensor([[ 6.,  8.],
        [10., 12.]])
Producto matricial:
 tensor([[19., 22.],
        [43., 50.]])
Media de a: 2.5
De NumPy a tensor: tensor([1, 2, 3])
```

```
De tensor a NumPy: [1 2 3]
Derivada de y en x=3: 8.0
```

Explicación

Si tu equipo tiene GPU NVIDIA con CUDA instalado, verás **Dispositivo disponible: cuda**; en otro caso, **cpu**. Ambos sirven para esta guía.
`requires_grad=True` le pide a PyTorch que registre las operaciones para luego derivar. Con `y.backward()` calcula dy/dx = 2x + 2, que en x = 3 vale **8**.
`.item()` convierte un tensor de un solo valor en un número de Python.

Paso 15. Entrenar un modelo con el bucle de entrenamiento

Crea **06_pytorch_regresion.py**. A diferencia de Scikit-learn (donde basta `fit`), en PyTorch **tú escribes el bucle de entrenamiento**. Estos 5 pasos se repiten en todas las redes neuronales:

N.°	Instrucción	Qué hace
1	`y_pred = modelo(X)`	Paso hacia adelante: el modelo hace una predicción.
2	`perdida = funcion_perdida(y_pred, y)`	Calcula el error de la predicción.
3	`optimizador.zero_grad()`	Borra los gradientes de la vuelta anterior.
4	`perdida.backward()`	Retropropagación: calcula cómo cambiar cada peso.
5	`optimizador.step()`	Actualiza los pesos para reducir el error.

Archivo: 06_pytorch_regresion.py

```
# 06_pytorch_regresion.py - Entrenar una regresión lineal con nn.Linear
import torch
from torch import nn

torch.manual_seed(0)

# Datos: y = 2x + 1 con algo de ruido
X = torch.linspace(0, 10, 100).unsqueeze(1)     # forma (100, 1)
y = 2 * X + 1 + torch.randn(100, 1) * 0.5

modelo = nn.Linear(in_features=1, out_features=1)
funcion_perdida = nn.MSELoss()
optimizador = torch.optim.SGD(modelo.parameters(), lr=0.01)

# Bucle de entrenamiento
for epoca in range(1, 1001):
    y_pred = modelo(X)                    # 1. paso hacia adelante
    perdida = funcion_perdida(y_pred, y)  # 2. calcular error
    optimizador.zero_grad()               # 3. limpiar gradientes
    perdida.backward()                    # 4. retropropagación
    optimizador.step()                    # 5. actualizar pesos
    if epoca % 200 == 0:
        print(f"Época {epoca}: pérdida = {perdida.item():.4f}")
```

```
peso = modelo.weight.item()
sesgo = modelo.bias.item()
print(f"Modelo aprendido: y = {peso:.2f}x + {sesgo:.2f}")
```

Resultado esperado (los decimales pueden variar ligeramente)

```
Época 200: pérdida = 0.3xxx
Época 400: pérdida = 0.2xxx
Época 600: pérdida = 0.2xxx
Época 800: pérdida = 0.2xxx
Época 1000: pérdida = 0.2xxx
Modelo aprendido: y = 2.0x + 1.0   (el peso debe quedar cerca de 2 y el sesgo cerca de 1)
```

Los datos se generaron con la fórmula **y = 2x + 1** más ruido; el modelo debe “descubrir” valores cercanos a 2 y a 1 sin que se los digamos.

Paso 16. Red neuronal para clasificar Iris (Scikit-learn + PyTorch)

Crea **07_pytorch_iris.py**. Usaremos Scikit-learn para preparar los datos y PyTorch para construir una red neuronal de dos capas (perceptrón multicapa).

Archivo: 07_pytorch_iris.py

```
# 07_pytorch_iris.py - Red neuronal (MLP) para clasificar Iris
import torch
from torch import nn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

torch.manual_seed(42)

# 1. Preparar datos con Scikit-learn
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
esc = StandardScaler()
X_train = esc.fit_transform(X_train)
X_test = esc.transform(X_test)

# 2. Convertir a tensores
X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

# 3. Definir la red neuronal
class RedIris(nn.Module):
    def __init__(self):
        super().__init__()
        self.capas = nn.Sequential(
            nn.Linear(4, 16),   # 4 entradas -> 16 neuronas
            nn.ReLU(),
            nn.Linear(16, 3),   # 16 neuronas -> 3 clases
        )

    def forward(self, x):
        return self.capas(x)

modelo = RedIris()
perdida_fn = nn.CrossEntropyLoss()
optimizador = torch.optim.Adam(modelo.parameters(), lr=0.01)

# 4. Entrenar
for epoca in range(1, 201):
    modelo.train()
    salida = modelo(X_train)
    perdida = perdida_fn(salida, y_train)
    optimizador.zero_grad()
    perdida.backward()
    optimizador.step()
    if epoca % 50 == 0:
        print(f"Época {epoca}: pérdida = {perdida.item():.4f}")

# 5. Evaluar
```

```
modelo.eval()
with torch.no_grad():
    predicciones = modelo(X_test).argmax(dim=1)
    exactitud = (predicciones == y_test).float().mean().item()
print(f"Exactitud en prueba: {exactitud:.3f}")

# 6. Guardar los pesos del modelo
torch.save(modelo.state_dict(), "red_iris.pth")
print("Pesos guardados en red_iris.pth")
```

Resultado esperado (los decimales pueden variar ligeramente)

```
Época 50: pérdida = 0.xxxx
Época 100: pérdida = 0.xxxx   (la pérdida debe ir bajando)
Época 150: pérdida = 0.0xxx
Época 200: pérdida = 0.0xxx
Exactitud en prueba: 0.9xx   (normalmente entre 0.90 y 1.00)
Pesos guardados en red_iris.pth
```

Explicación

La clase **RedIris** hereda de `nn.Module`. En `__init__` se definen las capas y en `forward` cómo pasan los datos por ellas.
ReLU es la función de activación que permite a la red aprender relaciones no lineales.
CrossEntropyLoss es la función de pérdida para clasificación; **Adam** es un optimizador que ajusta la velocidad de aprendizaje automáticamente.
`argmax(dim=1)` elige la clase con mayor puntaje. `torch.no_grad()` desactiva el cálculo de gradientes durante la evaluación (más rápido).
`state_dict()` guarda solo los pesos aprendidos en **red_iris.pth**.

Paso 17. Comparar Scikit-learn y PyTorch

Aspecto	Scikit-learn	PyTorch
Entrenamiento	Una línea: `modelo.fit(X, y)`	Bucle escrito por el programador
Estructura de datos	Arreglos de NumPy y DataFrames	Tensores
Uso de GPU	No	Sí (`.to("cuda")`)
Mejor para	Datos tabulares, prototipos rápidos	Redes neuronales profundas: imágenes, texto, audio
Guardar modelo	`joblib.dump()`	`torch.save(modelo.state_dict())`

Usar el depurador de VS Code

El depurador te permite detener el programa en una línea y revisar el valor de cada variable. Es la mejor forma de entender qué hace el código paso a paso.

1. Haz clic a la izquierda del número de línea donde quieres detenerte: aparecerá un **punto rojo** (punto de interrupción). También puedes usar `F9`.
2. Presiona `F5` y elige **Python Debugger > Python File** (archivo actual).

3. El programa se detiene en el punto rojo. En el panel **VARIABLES** (izquierda) verás los arreglos, tensores o DataFrames con sus valores.
4. Usa `F10` para avanzar línea por línea, `F11` para entrar a una función y `Shift + F5` para detener.
5. En la **CONSOLA DE DEPURACIÓN** puedes escribir expresiones, por ejemplo `df.head()`, y ver el resultado en ese momento.

> **Sugerencia**
> Coloca un punto de interrupción dentro del bucle `for epoca` del Paso 15 y observa en el panel **VARIABLES** cómo cambian `modelo.weight` y `perdida` en cada iteración.

Parte 4. Errores frecuentes y cómo resolverlos

Mensaje de error	Causa	Solución
`No se encontró Python; ejecutar sin argumentos para instalar desde Microsoft Store...`	Python no está instalado o no está en el PATH; Windows redirige el comando a la Microsoft Store.	Instala Python desde python.org marcando **Add python.exe to PATH**. Desactiva los alias en **Configuración > Aplicaciones > Configuración avanzada de aplicaciones > Alias de ejecución de aplicaciones** (python.exe y python3.exe). Cierra y vuelve a abrir VS Code.
`ModuleNotFoundError: No module named 'sklearn'`	Scikit-learn no está instalado en el entorno activo.	Instálalo con `pip install scikit-learn` (el nombre del paquete no es sklearn).
`ValueError: Expected 2D array, got 1D array`	`predict` recibió una lista simple.	Envía una lista de listas: `[[5.1, 3.5, 1.4, 0.2]]` o usa `reshape(-1, 1)`.
`NotFittedError`	Se llamó a `predict` antes de `fit`.	Entrena el modelo con `fit` antes de predecir.
`RuntimeError: mat1 and mat2 shapes cannot be multiplied`	Las dimensiones de la entrada no coinciden con la capa.	Verifica que `nn.Linear(4, ...)` reciba tensores con 4 columnas.
`RuntimeError: expected scalar type Long but found Float`	Las etiquetas de clasificación deben ser enteras.	Convierte y con `dtype=torch.long`.
La pérdida aparece como `nan` o sube sin parar	Tasa de aprendizaje (lr) demasiado alta.	Reduce lr (por ejemplo, de 0.1 a 0.01) o escala los datos.

Parte 5. Ejercicios propuestos

1. **Scikit-learn:** en el Paso 11 prueba con `n_neighbors` igual a 1, 3, 7 y 15. Anota la exactitud de cada uno en una tabla y explica cuál elegirías.

2. **Scikit-learn:** carga el dataset `load_wine()` y repite el Paso 12 agregando el modelo `RandomForestClassifier(random_state=42)`. ¿Cuál obtiene el mejor promedio?
3. **Scikit-learn:** modifica el Paso 13 para que los datos tengan más ruido (`rng.normal(0, 3, ...)`). ¿Cómo cambian el MAE y el R^2?
4. **PyTorch:** en el Paso 15 cambia `lr` a 0.001 y a 0.05. Describe qué pasa con la pérdida y con los valores aprendidos.
5. **PyTorch:** en el Paso 16 agrega una segunda capa oculta de 8 neuronas con `ReLU`. Compara la exactitud con la red original.
6. **Reto integrador:** carga los pesos guardados en `red_iris.pth` en un nuevo archivo (`modelo.load_state_dict(torch.load("red_iris.pth"))`) y predice la especie de 3 flores inventadas por ti.

Lista de cotejo (autoevaluación)

Criterio	Logrado	En proceso
Instalé Scikit-learn y PyTorch en un entorno virtual y verifiqué sus versiones.	☐	☐
Dividí datos en entrenamiento y prueba y escalé las características.	☐	☐
Entrené un clasificador y lo evalué con exactitud, matriz de confusión y reporte.	☐	☐
Comparé modelos con validación cruzada y guardé el mejor con joblib.	☐	☐
Entrené una regresión lineal e interpreté MAE y R^2.	☐	☐
Creé tensores, operé con ellos y calculé una derivada con autograd.	☐	☐
Programé el bucle de entrenamiento y entrené una red neuronal en PyTorch.	☐	☐
Usé el depurador de VS Code para observar el entrenamiento.	☐	☐